

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

DecisionAI is an AI-powered decision intelligence system designed to help retail SMEs diagnose sales decline using structured and unstructured data. The system leverages a Retrieval-Augmented Generation (RAG) architecture to combine internal business data with external market insights, enabling fast, explainable decision-making.

1.1. Purpose:

*This document outlines the **technical architecture, system design, and AI integration** behind DecisionAI.*

Key element covered are as followed:

1. Architecture:

- a. Backend hosted and deployed on Render:
 - i. Python RAG ingestion and retrieval logic
 - ii. Qdrant cloud for data vectorization storage
- b. Frontend hosted on vercel:
 - i. Next.js front end connected to backend via fastApi

2. Data Flows:

- a. User answer guided questions -> CSV upload -> data preprocessing -> embedding -> vector storage -> retrieval -> AI Generation

3. Model Process:

- a. End-to-end decision pipeline from user input to structured insights

4. Role of Reference:

- a. Serves as a technical blueprint for developers and evaluators.

1.2. Background:

Retail SMEs often rely on spreadsheets and manual analysis to diagnose business issues. This process is slow, error-prone, and lacks predictive capabilities.

1. Previous Version:

- a. Manual Excel-based analysis
- b. Static dashboards without reasoning
- c. No integration of external market data

2. Changes in Major Architectural Components

- a. Introduction of RAG pipeline
- b. Integration of LLM reasoning engine
- c. Use of vector database

3. New capabilities

- a. Context-aware reasoning
- b. Trade-off analysis
- c. Predictive insights
- d. Structured decision outputs

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
SME Owner	Upload data, receive insights	Fast, actionable decisions
Store Manager	Monitor performance	Clear recommendations
Development Team	Build system	Modular, scalable architecture
QA Team	Validate outputs	Reliable and structured responses

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Application
Architecture	Cloud-based, modular (RAG pipeline)

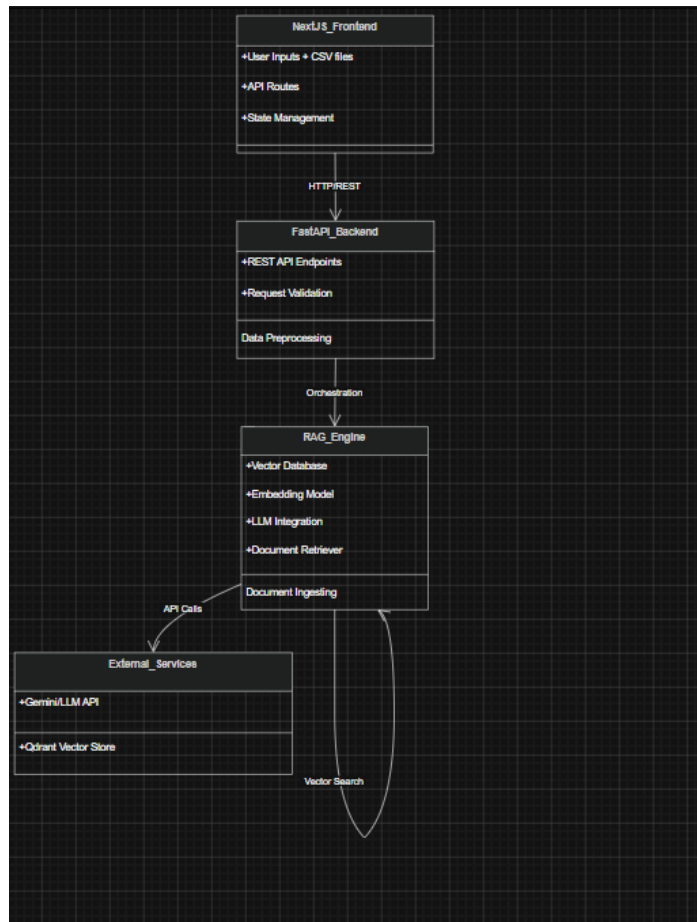
DecisionAI follows a **client-server architecture**:

- **Frontend** → User uploads CSV, answers guided questions and enters query
- **Backend API** → Handles data processing
- **Vector DB (Qdrant)** → Stores embeddings
- **LLM Layer** → Generates decision insights

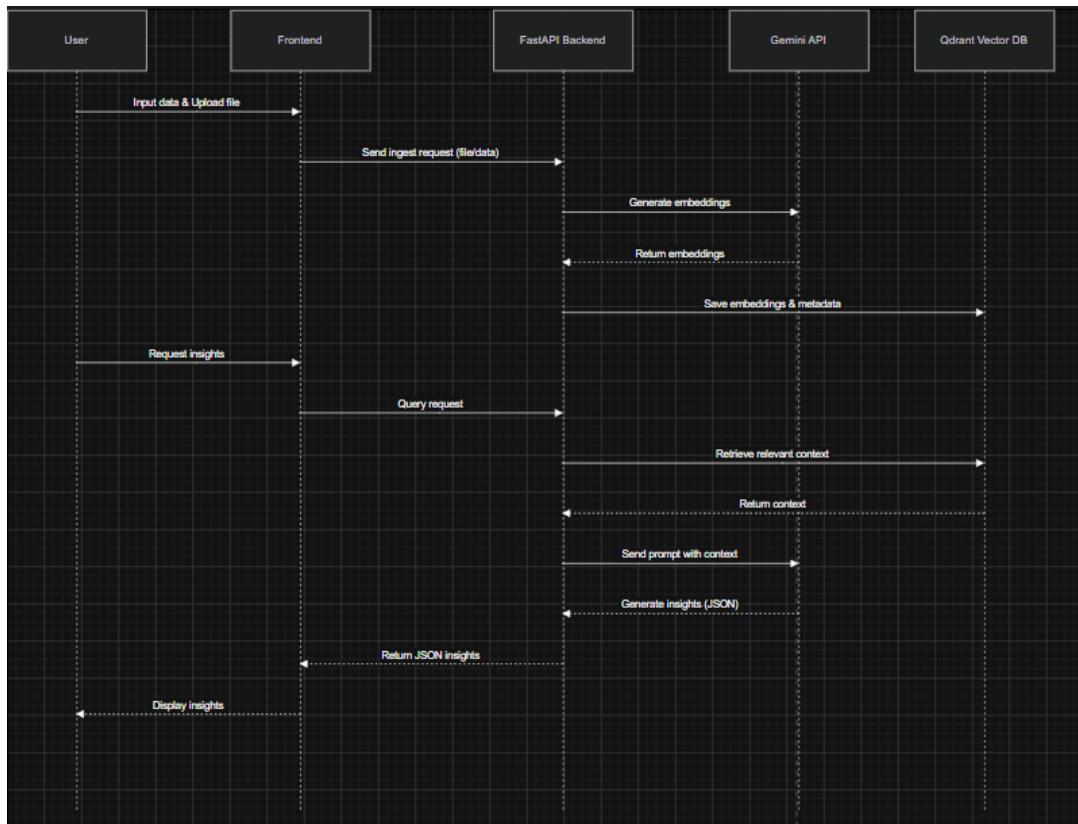
2.1.2 LLM as Service Layer

The LLM (Gemini by Google) is used as a central reasoning engine, not just a text generator.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. Technological Stack

2.2.1. Frontend : Next.js

2.2.2. Backend: Python FastAPI

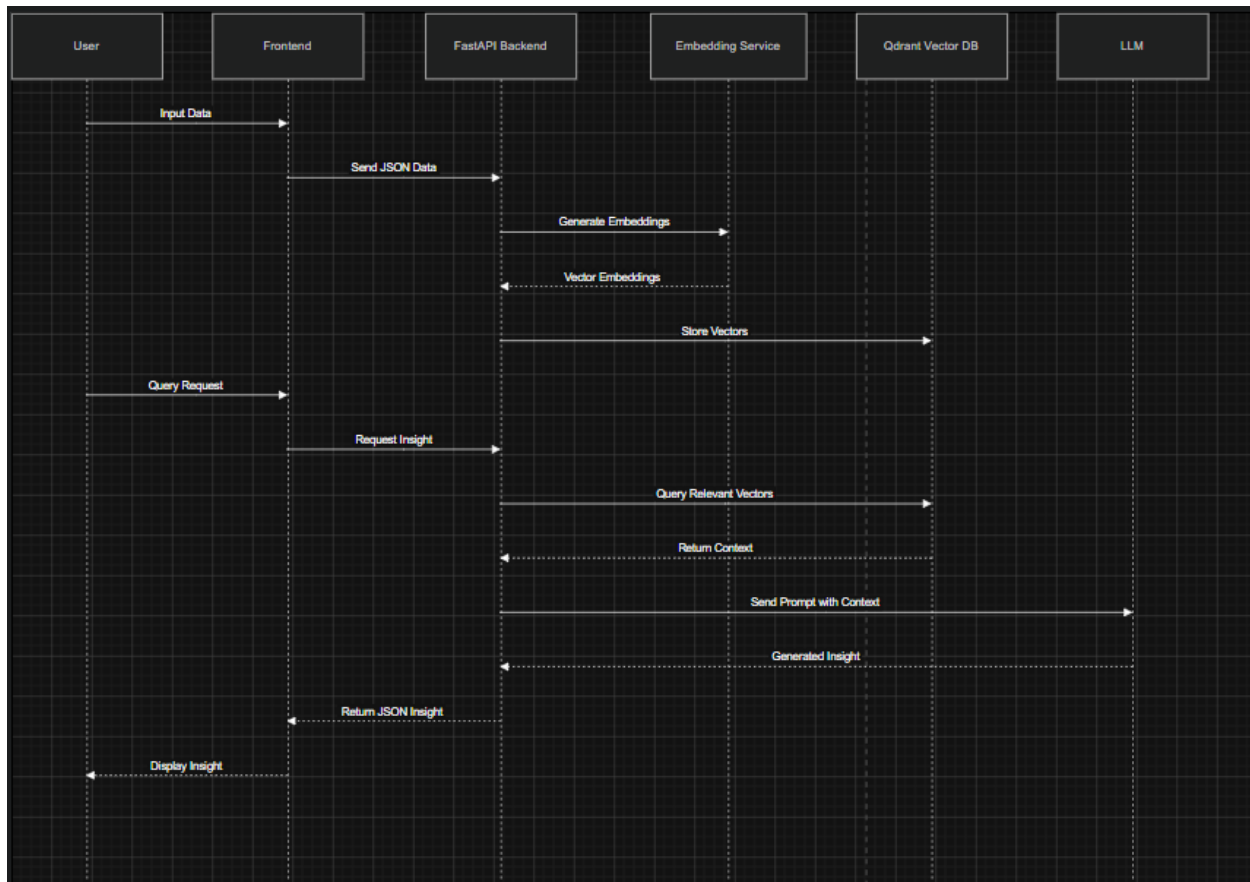
2.2.3. Database Qdrant (Vector DB)

2.2.4. Cloud/Deployment Render for backend and vercel for frontend

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

```

RAW_SCHEMA = {
  "type": "OBJECT",
  "properties": {
    "insight": {
      "type": "STRING"
    },
    "recommendations": {
      "type": "ARRAY",
      "items": {
        "type": "STRING",
        "description": (
          "Each recommendation must be written as a structured paragraph following this example format: "
          "<< Example Action Title >> Based on [data signals observed], [reasoning logic explaining why this matters]. "
          "As a result, [expected business impact with quantification where possible]. "
          "The response does not need to use the exact same sentence beginnings - "
          "this is only a guidance template to ensure all three elements are naturally present in the paragraph."
        )
      }
    }
  }
},

```

3.

```

"reasoning": {
  "type": "STRING"
},
"prediction": {
  "type": "STRING"
},

```

```

"tradeoffs": {
  "type": "OBJECT",
  "properties": {
    "optionA": {
      "type": "STRING"
    },
    "optionB": {
      "type": "STRING"
    },
    "verdict": {
      "type": "STRING"
    }
  },
  "required": [
    "optionA",
    "optionB",
    "verdict"
  ]
},

"confidence": {
  "type": "OBJECT",
  "properties": {
    "score": {
      "type": "NUMBER",
      "minimum": 0,
      "maximum": 100,
      "description": "Confidence score representing reliability of the prediction from 0 to 100 based on the confidence rules given."
    },
    "reason": {
      "type": "STRING",
      "description": "Explanation of why the confidence score was assigned, based on data completeness, quality, and reliability."
    }
  },
  "required": [
    "score",
    "reason"
  ]
}

```

4. Functional Requirements & Scope

4.1. Minimum Viable Product:

4.1.1.

#	Features	Description
1	CSV Data Upload	Users upload financial/transaction data
2	RAG Retrieval	Context retrieved from vector DB
3	AI Decision Engine	Generates insights and recommendations
4	Trade-off Analysis	Shows option A, option B, and final verdict
5	Structured Output	JSON-based explainable results

4.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	Handle multiple SME users from different industries	Modular backend
Reliability	Accurate outputs	Guided inputs, Schema validation, explainable and confidence level of the AI also in output.
Maintainability	Clean architecture	Backend and frontend are both deployed on separate servers and separated cleanly
Token Latency	< 10s response	Efficient prompt design
Cost Efficiency	Minimize token usage	RAG + pre saved datasets for repeated use

4.3. Out of Scope / Future Enhancements

- a. Real-time POS integration
- b. Real time Follow up and actual plan checklist for users
- c. Advanced forecasting models
- d. Advanced hybrid vector search for RAG

5. Monitor, Evaluation, Assumptions & Dependencies

5.1. Technical Evaluation:

5.1.1. Testing Strategy:

- Scenario-based validation:
 - o Upload declining sales data
 - o Verify AI identifies root causes
- Comparison:

- check confidence level of AI when csv is uploaded vs when not uploaded

5.1.2. **Emergency Handling:**

- Graceful error responses

5.1.3. **Priority Matrix:**

Priority	Condition	Action
P1	LLM failure	Retry/error message
P2	Slow response	Reduce context
P3	Minor inaccuracies	Log for improvement

5.2. **Monitoring:**

- Token Usage tracking
- API latency tracking

5.3. **Assumptions:**

- Users provide valid and related CSV data
- Internet connectivity is stable
- External APIs are available

5.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Gemini API	Core reasoning engine	High- Quota limits
Vercel	To host/deploy frontend website	Medium - Server being down causing users to have no GUI
Qdrant	Used for vector storage and contextual retrieval of data for RAG	Medium – There will be no contextual vector data storage
Python Backend	Processing	Low
Render	Deployment	Medium – Server being down causing Frontend server to not be able to access main processing backend

6. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

6.1. Project Timeline: *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

6.2. Team Members Role:

<i>Member</i>	<i>Role</i>	<i>Responsibility</i>
<i>Ong Hua Yu</i>	<i>Frontend lead</i>	<i>Frontend UI design documentation</i>
<i>Chuan He</i>	<i>Backend Lead</i>	<i>RAG pipeline, python backend, and integration</i>
<i>Ong Xin Yu</i>	<i>Documentation</i>	<i>documentation</i>

Recommendations:

- Add caching (Redis) for repeated queries
- Optimize prompt size for cost
- Add UI dashboard for insights visualization